

Mobile Code Security

Module 21

Duration: 26:38

Video: <https://www.youtube.com/watch?v=ccqp2LU6E7Q>

Course Coordinator - Dr Maninder Singh, Professor and Dean, Academic Affairs, Thapar University, Patiala

Subject Expert - Dr Navdeep Singh, Department of Computer Engineering, Punjabi University, Patiala

Auto-generated YouTube transcript for personal study. Spoken wording belongs to the original lecture. Follow the video link above; this file is not an official CEC/SWAYAM publication.

0:03 [Music]

0:10 my dear students hello I welcome you all in our series of lectures on cyber

0:17 security in the previous lecture we studied about Information Security Management today we are going to discuss

0:26 mobile code security and the various issues associated with it so let's start with

0:33 it the mobile code Paradigm and Compromises programs that can be executed on one or several hosts other than the

0:42 one that they originate from mobility of such programs implies some built-in capability for each piece of code to

0:52 travel smoothly from one host to another a mobile code is associated with at least two parties is producer and its

1:01 1 minute, 1 second consumer the consumer being the host that runs the code mobile code systems range from simple applets to intelligent

1:09 software agents these systems offer several advantages over the more traditional distributed computing approaches

1:18 flexibility in software design beyond the well-established object-oriented Paradigm and bandwidth

1:25 optimization just to name two of them as usual increased flexibility comes with a cost that is increased

1:34 vulnerability in the face of malicious intrusion scenarios akin to internet possible vulnerabilities with mobile

1:41 code fall in one of the two categories attacks performed by a mobile program against the remote host on which the

1:49 program is executed as with malicious applets or ActiveX programs and the less

1:56 classical category of attacks due to the subversion of the mobile code and its data by the remote execution

2:05 environment now let's discuss mobile code security threats a first security threat appears

2:12 if the mobile code generated by a malicious Outsider attacks the environment

where it is

- 2:20 executed this problem bear some similarity with Trojan horses but mobile code brings up other issues such as
- 2:29 mobile code aims at transparency Automation and a wider scale of execution moreover vulnerabilities of
- 2:38 the environment are not the only Targets in addition to this relatively classical scenario mobile code
- 2:46 introduces a completely new problem such as the host itself might be the malicious party trying to subvert the
- 2:56 mobile when protecting a host from potentially malicious code code Mobility imposes some of the security
- 3:03 features such as number one host and mobile code be of separate identities
- 3:10 therefore the mobile code's origin must be authenticated number two mobile code is exposed through the network hence the
- 3:19 host must verify the Integrity of the mobile code it just received number three the host does not generate the
- 3:28 mobile code but another party does consequently the actions it performs must be limited through access control
- 3:37 and checked through semantic verification when protecting a mobile code from a potentially malicious host
- 3:45 code Mobil implies that the program will be run under total control of the host this
- 3:51 means the threats such as spoofing through impersonation of code owner theft and secrecy violation ation
- 4:00 through unauthorized disclosure Integrity violation through subversion of code semantics to prevent
- 4:07 all these cases data segments as well as source semantics must be protected next is protection of a host
- 4:15 from a mobile code the subject of host protection is very important the first answer in this
- 4:23 area was to Simply limit the functionality of the execution environment in order to limit the vulnerabilities
- 4:30 techniques for protection of host now evolve along two directions a mobile core infrastructure that is gradually enhanced with
- 4:39 authentication data integrity and access control mechanisms verification of mobile code
- 4:47 semantics there are various techniques detailing these aspects let us discuss them one by one number one sand
- 4:56 boxing sand boxing consists in running a mobile code in a restricted environment called The

5:02 Sandbox and otherwise untrusted mobile code can be executed without worrying in the

5:10 sandbox a sand box can be characterized by two different mechanisms number one it confines code either through type

5:18 checking language Properties or the use of protection domains to prevent the subversion of trusted code and number

5:26 two it enforces a fixed policy for the execution of code this approach is especially

5:34 illustrated by the early Java jdk 1.0 where it was used in order to enable

5:41 applets available anywhere on the internet to run within a browser the major drawback of sandboxing

5:49 is due to the fact that applications running in such a restrictive environment are themselves seldom useful

5:57 next is digital shrink wrap another approach to protect an execution environment against potentially

6:05 malicious mobile code is to authenticate the mobile code before it is actually

6:12 executed in this approach the producer of the code is required to sign in and the code consumer verifies the signature

6:20 of the producer before using it although it is not possible to decide whether a given piece of mobile code

6:28 contains malicious code code one can at least determine whether it is authentically coming from its claimed

6:36 Source Microsoft has proposed this approach in its authentic code technology note that the two approaches

6:44 sandboxing and digital shrink WS can be combined to provide more sophisticated protection

6:51 schemes Sun Microsystems has combined both in its new Java 1.2 security model

7:00 number three proof carrying code recently neula and Lee from K melon University propose a new technique to

7:09 protect a host from a malicious mobile code the technique is called proof carrying code

7:16 PCC PCC enables a host to determine automatically and with certainty that a

7:23 program code provided by another system is safe to install and execute

7:30 the basic idea of PCC is that the code producer is required to provide an encoding of a proof that his her code

7:39 adheres to the security policy specified by the code consumer the proof is encoded in a form

7:48 that can be transmitted digitally therefore the code consumer can quickly validate the code using a simple

7:57 automatic and reliable proof checking process next is steps for generating and

8:04 verifying the proof carrying code also called PCC a typical PCC session requires five

8:12 steps to generate and verify the PCC step number one apcc session starts with the code

8:21 producer preparing the untrusted code to be sent to the code consumer the producer adds annotation to

8:29 the code which can be done manually or automatically by a tool such as a certifying

8:36 compiler these annotations contain information that helps the code consumer to understand the safety relevant

8:44 properties of the code the code producer then sends the annotated code to the code consumer to

8:52 execute it step number two the code consumer performs a fast but detailed inspection

9:00 of the annotated code this is accomplished using a program called VC gen which is one of the component of the

9:08 consumer defined safety policy VC gen performs two tasks first it checks simple safety properties of

9:16 the code for example it verifies that all immediate jumps are within the code segment

9:23 boundaries second VC gen watches for instructions whose execution might violate the safety

9:32 policy when such an instruction is encountered VC gen emits a predicate that expresses the conditions under

9:40 which the execution of the instruction is safe the collection of the verification conditions together with some control

9:48 flow information make up the safety predicate and a copy of it is sent to the proof

9:55 producer step number three a on receiving the safety predicate the proof producer attempts to

10:03 prove it and in the event of success it sends an encoding of formal proof back to the code

10:11 consumer because the code consumer does not have to trust the proof producer any system can act as a proof

10:20 producer step number four then the code consumer performs a proof validation this phase is performed using a proof

10:29 proof Checker the proof Checker verifies that each inference step in the proof is

10:36 a valid instance of one of the axioms and inference rules specified as part of the safety

10:43 policy in addition the proof Checker verifies that the proof proves the same safety predicate generated in Step

10:52 number two step five finally after the executable code has passed both the VC
11:00 gen checks and the proof check it is trusted not to violate the safety policy it can
thus be safely installed for
11:09 execution without any further need for runtime checking next is code
11:16 signing code signing is the process by which a code is digitally signed by the code
producer in order to assure strong
11:26 authentication and integrity of the code to the code consumer this model was
first introduced
11:33 by Microsoft within the active X framework Java jtk 1.1 also follows the
11:40 code signing model with so-called applets upon re of an applet with a valid
signature the code consumer Java
11:49 virtual machine executes the applet like a trusted piece of code authorizing it
11:56 to access all features available in Java an applet without a proper signature is
12:03 run in a sandbox as in the previous version of the jdk there is more to scaring a
host from
12:10 a malicious mobile code than just making sure that this program has been correctly
signed by someone on the
12:18 internet the letter simply does not imply that the code signer must be trusted
unrestrictedly moreover the presumption
12:27 underlying code signing that users can decide whether to run a program based on a
signature has questionable
12:36 validity a later drawback of code signing lies in the rudimentary form of Access
Control provided a sign code is
12:44 either granted full access to the resources of the code consumer or not executed at
12:51 all this choice is Left To the End user who even without administrator privileges
can put the entire security
13:00 at risk number five access control the only way to limit this kind
13:08 of security threat is to enable more complex Access Control schemes in order to
limit the impact of an
13:15 attack this can be seen as the refinement of a monolithic sandbox policy into
smaller application specific
13:25 policies the identity of the signer of a code AS Des described before also helps to
further refine the execution policy
13:34 definition provided that it can be established by some mean like a public key
infrastructure for instance Java GDK
13:44 1.2 security model follows this scheme and thus permits the definition of finer
13:50 Grant security policies more suited to executing an untrusted mobile code even
13:57 more recently Sun releas the Java authentication and authorization services or Jazz
which aims at

14:05 integrating the identity of the user running the mobile code in the definition of the access control compared with sandboxing and

14:14 code signing the access control model has the best of both worlds the actions performed by a mobile

14:21 code can be stricted to some resources while it permits at the same time to write and run really useful

14:30 software yet the enforcement of the excess scheme has a cost since it is

14:37 performed dynamically at runtime number six program checking checking a mobile

14:44 code means to perform a verification on the code structure or on the code Behavior as it is run and modifying in

14:53 Consequence the status of the code for instance from trusted to untrusted checking also involves the notion of

15:02 verification against a given security policy sandboxes already exercise some form or another of rudimentary program

15:10 check either statically for instance to ensure that operants of an instruction are of the correct type or dynamically

15:20 for example to locate any access to a protected resource a newer approach to host

15:27 protection is to St Ally type check the mobile code the program is then run without any expensive runtime

15:35 checks this approach is for instance taken in the proof carrying code scheme and even to some extent in Java's

15:44 virtual machine for safety checks the proof carrying code scheme or PCC is a good instance of the static

15:52 program checking model in this scheme a predefined security policy is defined in terms of a

16:00 logic the host first asks to be sent a proof that the code respects the policy

16:06 before he actually agrees to run it the code producer then sends the program and an accompanying proof using a set of

16:15 sound XMS and rewriting rules as defined by The Chosen logic and shared by the code producer and the code

16:23 consumer after receiving the code the host can then check the program with a guide guidance of the proof this can be

16:31 seen as a form of type checking of the program since the proof is directly driven from

16:37 it the most interesting part of PCC is that type checking the proof is relatively obvious compared to proving

16:45 the program and does not impose much computational burden on the code consumer moreover since the proof is not

16:54 the program PCC already permits to express complex safety properties it seems promising for

17:02 security properties as well however automating the proof generation is still a non-trivial

17:09 problem and to date proofs generally have to be generated by hand next is protection of a mobile code from a

17:17 malicious host the problem of protection from a malicious host has been studied only recently and is intrinsically more

17:26 difficult because the environment gets a total control over the mobile code otherwise host protection would not be

17:35 possible the approaches used for that purpose can be classified along two criteria number one data versus code

17:44 protection and number two Integrity or confidentiality based data protection is essentially

17:52 interested in protecting mobile agents roaming in the internet an agent might encounter a

18:00 malicious host occasionally who will try to brainwash the agent for instance on the other hand code

18:08 protection issues address a more systematic form of maliciousness in which the environment where the mobile

18:15 code runs cannot be trusted now let's learn more about data protection mobile or roaming agents are

18:25 form of mobile code especially publicized in electronic Commerce but usual data protection techniques are not

18:33 well adapted for protecting their data mobile agents are used to perform a

18:39 function on behalf of the user like for instance collect data about a product

18:45 from several merchants or perform a transaction a good example might be the so-called comparison shopping for

18:54 instance an agent looks for the lowest priced book among several retailers in that kind of scenario it is

19:03 necessary to know whether the data collected have been changed or not if the itinerary fixed for the chain

19:12 of operations has been followed in an auction scenario it might also be necessary to prevent a host from

19:20 exploiting the offers made by the previous bidders the Integrity of the data collected by a mobile agent might be

19:28 protected using a cryptographic technique a first possible technique is simply to digitally sign the result the

19:37 problem with this approach is that the size of the agent grows linearly as it gathers results a more efficient approach in terms of computational complexity and space is

19:45 to use hash chaining to achieve

19:54 forward Integrity Integrity against deletion or modification of parts of the collector
20:00 data in different host is achieved until the first malicious host a partial result
authentication code is small
20:09 value ensures the Integrity of all the offers this solution is extended with a
20:16 cryptographic technique that allows host to update their previous data without
increasing the space
20:22 requirements these techniques have to be enhanced with digital signatures if non
repudiation is
20:29 required on its itinerary from host to host the agent carries an increasing amount
of data that might be exploited
20:38 by a malicious host their confidentiality can be achieved at each step of the agent
itinerary by a simple
20:46 and siring on the current host before the agent moves elsewhere with RSA based
encryption the
20:53 security will be ensured but the data carried by the agent can be very small the size
of the data padding will then be
21:01 21 minutes, 1 second excessive sliding encryption and aims at retaining an
equivalent security using a
21:07 large key while at the same time taking into account The Limited storage of an
agent sliding encryption is aimed at
21:16 conserving space rather than time which might be of importance for agents that
collect small amounts of data on many
21:24 different host next is integrity of computer ation a mobile code might be rendered
for
21:32 tolerant by performing the work to be done several times this applies in the case of
mobile
21:39 agents the same mobile agent code can be distributed to replicate host and the
result is discovered through a vote
21:49 Alterna and more convincingly it is also feasible to replicate agents and slightly
alter their behavior
21:56 orary in order to sport quot a malicious host a mobile code might also be
22:04 protected using a cryptography based Integrity proof or rather a hint of the
Integrity of its
22:11 computation this proof intends to guarantee that the computation was done
according to the instructions of the mobile code the trace of the program
22:19 computation is used as a form of proof showing how the result was obtained in
order to make the proof
22:27 checking process faster this proof is transformed into a holographic proof a
holographic proof system is a

22:35 system for writing and checking proofs in which one can probabilistically check the validity of a proof by examining

22:44 only a few of its bits moreover this checking can be performed in poly logarithmic

22:51 time the transformation of the proof into holographic proof is computationally intensive but it is the

22:58 host where the agent is executing and not the verifier which performs the computation another problem arises the

23:07 validity of the holographic proof is only preserved in so far as the bits of the proof carried by the verifier remain

23:14 unknown to the host that performed the computation this could be ensured if the holographic proof could be send to the

23:23 verifier host but a holographic proof is even bigger than the proof it was obtained

23:30 from this problem is solved by using a cryptographic private information retrieval technique with this technique the

23:37 holographic proof remain remain on the host which computed it and can be queried without the host knowledge of the queried

23:45 bits the result of these hidden queries and the computation result is finely smaller than the original proof itself

23:54 then comes privacy of computation a mobile file code might be written in order to rely on temper proof

24:02 hardware for instance a smart card as a skure kernel for executing secret functions and keeping secret data in

24:10 that case the achieved services are integrity and partial confidentiality of the program a mobile code might finally be

24:20 protected algorithmically by hiding the computed function this results in a standalone code without additional Hardware

24:29 achieving confidentiality of the computation this technique is known as function hiding in the function hiding

24:37 scheme IF function f is encrypted into EF by its Stander then EF is run on the

24:43 potentially malicious site with X as an input the result of EFX is returned and

24:50 decrypted yielding the result of FX computations involving matrices can be hidden and all functions that can be

25:00 represented by a matrix as well like for example combinatorial Boolean

25:07 circuits with this our today's lecture is over today we learned that mobile code is generally advocated for bringing

25:14 substantially increased flexibility and extensibility but these do not come without a price on security as we saw a

25:23 good level of host protection is achievable there is currently a a trend towards enabling finer grain Access

25:31 Control schemes thanks to this mobile CES will probably find more useful applications in the near

25:39 future semantic based analysis might also provide a new means of securing host without compromising

25:49 performances on the other hand the protection of a mobile code against a Malaya host is still an open search

25:56 topic anyhow it can be readily observed that non- cryptographic techniques are generally not sufficient to protect a

26:04 mobile code thank you and have a nice day

26:09 [Music]